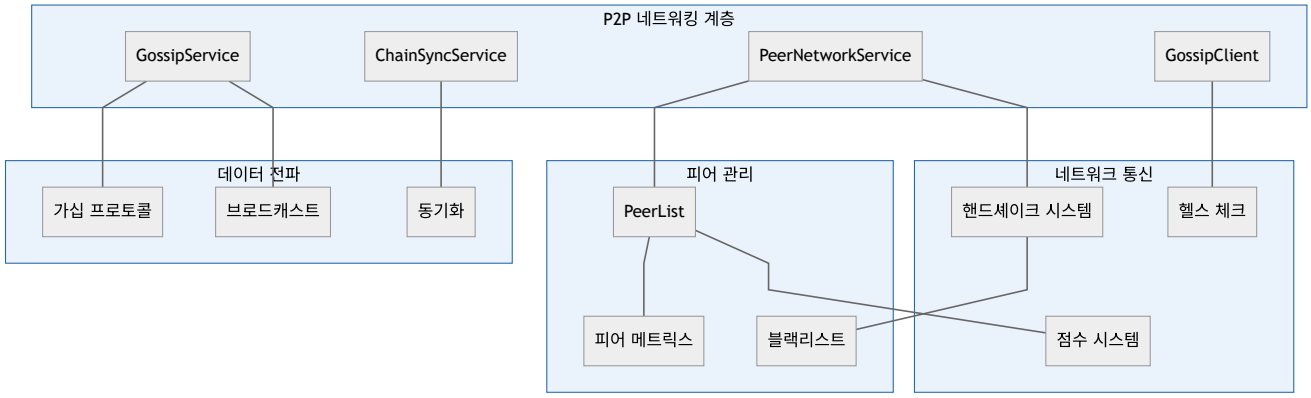
P2P 네트워크 기본 구조

개요

Irys 체인의 P2P 네트워킹 시스템은 분산된 노드들 간의 효율적인 통신과 데이터 동기화를 위해 설계된 고성능 네트워킹 아키텍처입니다. 이 시스템은 피어 발견, 연결 관리, 그리고 네트워크 토폴로지 유지를 통해 안정적이고 확장 가능한 블록체인 네트워크를 구축합니다.

아키텍처 개요



핵심 구성 요소

1. PeerNetworkService

역할: 피어 네트워크의 중앙 관리자로서 피어 발견, 연결 관리, 그리고 네트워크 상태 유지를 담당합니다.

주요 기능:

- 피어 발견 및 연결 설정
- 핸드셰이크 프로세스 관리
- 피어 점수 시스템 운영
- 네트워크 상태 모니터링

```
pub  struct PeerNetworkService<A, R>

where

A: ApiClient,

R: Handler<RethPeerInfo, Result = eyre::Result<()>> + Actor<Context = Context<R>

{

/// 노드 데이터베이스 참조

db: DatabaseProvider,

/// 피어 목록 관리

peer_list: PeerList,

/// 현재 진행 중인 공지 작업

currently_running_announcements: HashSet<SocketAddr>,

/// 가십 클라이언트

gossip_client: GossipClient,

/// API 클라이언트

irys_api_client: A,

/// 체인 ID
```

```
chain_id: u64,
/// 피어 주소 정보
peer_address: PeerAddress,
}
```

2. 피어 발견 메커니즘

2.1 초기 피어 발견

신뢰할 수 있는 피어 부트스트래핑:

```
async fn trusted_peers_handshake_task(
peer_service_address: Addr<Self>,
trusted_peers_api_addresses: HashSet<SocketAddr>,
) {
for peer_api_address in trusted_peers_api_addresses {
match peer_service_address
.send(NewPotentialPeer::force_announce(peer_api_address))
.await
{
Ok(()) => {}
Err(mailbox_error) => {
error!("Failed to send NewPotentialPeer message: {:?}", mailbox_error);
}
};
```

```
}

}
```

2.2 동적 피어 발견

피어 응답을 통한 확장:

- 핸드셰이크 성공 시 새로운 피어 목록 수신
- 응답에서 받은 피어들을 무작위로 섞어 처리
- 리소스 고갈 방지를 위한 피어 수 제한

```
PeerResponse::Accepted(accepted_peers) => {
let mut peers = accepted_peers.peers;
peers.shuffle(&mut rand::thread_rng());
let limit = peers_limit();
if peers.len() > limit {
peers.truncate(limit);
}
// 새로운 피어들을 처리 대기열에 추가
}
```

연결 관리 시스템

1. 핸드셰이크 프로세스

1.1 버전 요청 생성

```
fn create_version_request(&self) -> VersionRequest {
let signer = self.config.irys_signer();
let mut version_request = VersionRequest {
address: self.peer_address,
chain_id: self.chain_id,
user_agent: Some(build_user_agent("Irys-Node", env!("CARGO_PKG_VERSION"))),
..VersionRequest::default()
};
signer
.sign_p2p_handshake(&mut version_request)
.expect("Failed to sign version request");
version_request
}
```

1.2 동시 핸드셰이크 제한

글로벌 세마포어를 통한 리소스 관리:

```
static HANDSHAKE_SEMAPHORE: std::sync::OnceLock<std::sync::Arc<tokio::sync::Sema
std::sync::OnceLock::new();

// 동시 핸드셰이크 수 제한
let _permit = semaphore.acquire().await.expect("semaphore closed");
```

2. 연결 상태 관리

2.1 헬스 체크 시스템

비활성 피어 상태 확인:

```
ctx.run_interval(INACTIVE_PEERS_HEALTH_CHECK_INTERVAL, |act, ctx| {
let  inactive_peers: Vec<(Address, PeerListItem)> = act.peer_list.inactive_peers
for (mining_addr, peer) in  inactive_peers {
let  client = act.gossip_client.clone();
let  fut = async  move { client.check_health(peer_address).await }
.into_actor(act)
.map(move |result, act, _ctx| match  result {
Ok(true) => {
act.increase_peer_score(&mining_addr, ScoreIncreaseReason::Online);
}
Ok(false) => {
act.decrease_peer_score(&mining_addr, ScoreDecreaseReason::Offline);
}
Err(e) => {
error!("Failed to check health of peer {:?}: {:?}", mining_addr, e);
}
});
ctx.spawn(fut);
}
```

```
});
```

네트워크 토폴로지 및 라우팅

1. 피어 선택 전략

1.1 활성 피어 우선순위

점수 기반 피어 선택:

- 높은 점수를 가진 피어 우선 선택
- 신뢰할 수 있는 피어와 일반 피어 구분
- 무작위 선택을 통한 네트워크 다양성 확보

1.2 로드 밸런싱

브로드캐스트 피어 분산:

```
const MAX_PEERS_PER_BROADCAST: usize = 5;


// 피어를 무작위로 섞고 제한된 수만 선택
peers.shuffle(&mut rand::thread_rng());
let  n = std::cmp::min(MAX_PEERS_PER_BROADCAST, peers.len());
```

2. 네트워크 파티션 대응

2.1 다중 경로 통신

여러 피어를 통한 데이터 요청:

```
// 최대 5번의 시도로 여러 피어에서 데이터 요청
for attempt in 1..=5 {
for peer in &peers {
match self.pull_data_and_update_the_score(peer, data_request.clone(), peer_list
Ok(Some(data)) => return Ok((*address, data)),
Ok(None) => continue, // 피어가 데이터를 가지고 있지 않음
Err(err) => continue, // 다음 피어 시도
}
}
}
```

피어 평가 및 관리 메커니즘

1. 점수 시스템

1.1 점수 증가 요인

```
pub enum ScoreIncreaseReason {
Online, // 온라인 상태 확인
SuccessfulData, // 성공적인 데이터 전송
ValidResponse, // 유효한 응답
}
```

1.2 점수 감소 요인

```
pub enum ScoreDecreaseReason {
Offline, // 오프라인 상태
InvalidData, // 잘못된 데이터
NetworkError, // 네트워크 오류
}
```

2. 블랙리스트 관리

2.1 임시 차단 시스템

연속 실패 기반 차단:

```
static BLOCKLIST_UNTIL: std::sync::OnceLock<
std::sync::Mutex<HashMap<SocketAddr, std::time::Instant>>,
> = std::sync::OnceLock::new();

// 차단된 피어 확인
if let Some(until) = blocklist_until()
.lock()
.expect("blocklist_until mutex poisoned")
.get(&msg.api_address)
.copied()
{
if std::time::Instant::now() < until {
```

```
debug!("Peer {:?} is blacklisted until {:?}, skipping", msg.api_address, until);
return;
}
}
```

2.2 지수 백오프

실패 횟수에 따른 대기 시간 증가:

```
static HANDSHAKE_FAILURES: std::sync::OnceLock<std::sync::Mutex<HashMap<SocketAd
std::sync::OnceLock::new();

// 연속 실패 횟수 추적 및 백오프 계산
```

성능 최적화

1. 동시성 관리

액터 모델 기반 비동기 처리:

- 각 피어 작업을 독립적인 태스크로 처리
- 논블로킹 I/O를 통한 높은 처리량 달성
- 백프레셔 메커니즘으로 리소스 보호

2. 메모리 효율성

캐시 및 상태 관리:

- LRU 캐시를 통한 메모리 사용량 제한
- 주기적인 상태 플러시로 데이터베이스 동기화
- 불필요한 연결 정리

3. 네트워크 효율성

대역폭 최적화:

- 압축된 메시지 형식 사용
- 중복 데이터 전송 방지
- 적응형 타임아웃 설정

보안 고려사항

1. 인증 및 권한 부여

디지털 서명 기반 인증:

- 모든 핸드셰이크 메시지에 디지털 서명 적용
- 체인 ID 검증을 통한 네트워크 격리
- 사용자 에이전트 검증

2. DDoS 방어

레이트 리미팅 및 리소스 보호:

- 동시 연결 수 제한
- 핸드셰이크 빈도 제한
- 리소스 고갈 공격 방지

3. 데이터 무결성

메시지 검증:

- 모든 수신 데이터의 무결성 검증
- 잘못된 데이터 전송 시 피어 점수 감소
- 악의적인 피어 자동 차단

모니터링 및 디버깅

1. 로깅 시스템

구조화된 로그:

- 피어 연결 상태 추적
- 네트워크 이벤트 기록
- 성능 메트릭 수집

2. 메트릭스

네트워크 상태 지표:

- 활성 피어 수
- 평균 응답 시간
- 데이터 전송 성공률
- 네트워크 처리량

이 P2P 네트워크 기본 구조는 Irys 체인의 분산 특성을 지원하며, 확장성과 안정성을 모두 고려한 설계를 제공합니다.